Computer Assignment Chapter 8: Monte Carlo Estimation vs. Black-Scholes Formula

Matteo Pinzani

March 2026

1 Introduction

The goal of this assignment is to estimate the price of a European Call option using Monte Carlo simulation under the continuous-time risk-neutral measure, and to compare this empirical estimate with the exact analytical solution provided by the Black-Scholes formula.

2 Theoretical Explanation

2.1 Monte Carlo Estimation

Under the continuous-time risk-neutral framework, the terminal stock price $S(T)$ is:

$$S(T) = S(0)e^{(r-\frac{1}{2}\sigma^2)T+\sigma W^*(T)} \tag{1}$$

where $W^*(T)$ is a random variable following a normal distribution with mean 0 and variance T under the risk-neutral measure.

The value of a European Call option at time $t = 0$, denoted as $C_E(0)$, is the discounted expected value of its terminal payoff:

$$C_E(0) = e^{-rT}\mathbb{E}_*[\max(S(T) - X, 0)] \tag{2}$$

By simulating a large number of independent terminal prices $S_i(T)$ using standard normal variables scaled by $\sqrt{T}$, we can approximate using the sample mean.

2.2 The Black-Scholes Formula

The continuous-time limit of the binomial model produces the analytical Black-Scholes formula for a European Call option (Theorem 8.11).

$$C_E(0) = S(0)N(d_+) - Xe^{-rT}N(d_-) \tag{3}$$

where $N(x)$ is the cumulative standard normal distribution function, and:

$$d_+ = \frac{\ln(S(0)/X) + (r + \frac{\sigma^2}{2})T}{\sigma\sqrt{T}}$$
$$d_- = d_+ - \sigma\sqrt{T}$$

3 Parameter Selection

For the purpose of this numerical experiment, the market parameters are:

- Initial stock price, $S(0) = 100$
- Strike price, $X = 100$
- Risk-free interest rate (continuously compounded), $r = 0.05$ (5%)
- Volatility, $\sigma = 0.20$ (20%)
- Time to maturity, $T = 1$ year
- Number of Monte Carlo simulations, $M = 1,000,000$

4 Results and Comparison

The simulation was conducted using MATLAB. Generating $M = 10^6$ sample paths provided a robust distribution.

- **Exact Black-Scholes Price**: 10.4506
- **Monte Carlo Estimate**: 10.4581
- **Monte Carlo Standard Error**: 0.0147
- **Absolute Error**: 0.0075

5 Conclusion

The results demonstrate a near perfect convergence between the Monte Carlo estimation and the analytical Black-Scholes formula. The absolute error is quite small and falls well within the confidence interval suggested by the standard error. This computational exercise successfully validates the theoretical link between simulating asset paths under continuous Black-Scholes dynamics and the exact analytical pricing formula derived in Chapter 8.

6 MATLAB Code

The following script was used to generate the results presented in this report.

```
% Parameters Selection
S0 = 100;        % Initial stock price
X = 100;         % Strike price
r = 0.05;        % Risk-free interest rate
sigma = 0.20;    % Volatility
T = 1;           % Time to maturity (years)
M = 1000000;     % Number of Monte Carlo simulations

% ---------------------------------------------------------
% 1. Black-Scholes Exact Formula
% ---------------------------------------------------------
d1 = (log(S0 / X) + (r + 0.5 * sigma^2) * T) / (sigma * sqrt(T));
d2 = d1 - sigma * sqrt(T);

% Calculate N(d1) and N(d2) using the built-in error function (erf)
N_d1 = 0.5 * (1 + erf(d1 / sqrt(2)));
N_d2 = 0.5 * (1 + erf(d2 / sqrt(2)));

% Calculate Black-Scholes Call Price
C_BS = S0 * N_d1 - X * exp(-r * T) * N_d2;

fprintf('Exact Black-Scholes Price: %.4f\n', C_BS);

% ---------------------------------------------------------
% 2. Monte Carlo Estimation
% ---------------------------------------------------------
% Generate M random numbers from standard normal distribution N(0,1)
Z = randn(M, 1);

% W*(T) has variance T, so we multiply Z by sqrt(T)
W_T = sqrt(T) * Z;

% Simulate the terminal stock prices S(T) under risk-neutral measure
S_T = S0 * exp((r - 0.5 * sigma^2) * T + sigma * W_T);

% Calculate the payoffs
Payoffs = max(S_T - X, 0);

% Calculate the Monte Carlo estimate (discounted average payoff)
C_MC = exp(-r * T) * mean(Payoffs);

% Calculate Standard Error for the Monte Carlo estimate
SE = std(exp(-r * T) * Payoffs) / sqrt(M);

fprintf('Monte Carlo Estimate: %.4f\n', C_MC);
fprintf('Monte Carlo Standard Error: %.4f\n', SE);
fprintf('Absolute Error: %.4f\n', abs(C_BS - C_MC));
```